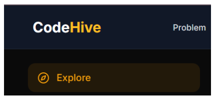
Reading Files in C

Official CodeHive Developer Guide



Reading data is a fundamental part of file handling in C, achieved through the <stdio.h> library.

1. FILE Pointer

All file access is handled via a pointer to a `FILE` structure.

```
FILE *fp;
```

2. Open File in Read Mode

To read an existing file, use `fopen()` with the `"r"` mode.

```
fp = fopen("data.txt", "r");
```

- **"r"**: Opens the file for reading only.
- **Critical:** If the file does not exist, `fopen` returns `NULL`.

3. Reading Data from File

a) Using fscanf()

Reads formatted input (like strings, integers) until whitespace is encountered.

```
fscanf(fp, "%s", str);
```

b) Using fgets()

Reads a whole line or a string of specific size. Safer for multi-word text.

```
fgets(str, sizeof(str), fp);
```

c) Using fgetc()

Reads a single character from the current position.

```
char ch = fgetc(fp);
```

4. Checking End of File (EOF)

To read until the end of the file, use the `feof()` function or check against `EOF`.

```
while (!feof(fp)) {
    // Process data
}
```

5. Closing the File

Always close your file to free up system memory.

```
fclose(fp);
```

Simple Example

```
#include <stdio.h>

int main() {
    FILE *fp;
    char ch;

    fp = fopen("file.txt", "r");

    if (fp == NULL) {
        printf("Error: File not found\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF) {
        printf("%c", ch);
    }
```

```
    fclose(fp);
    return 0;
}
```

Key Points (Very Important ⭐)

- Library: **<stdio.h>**
- Mode **"r"** is for reading only.
- If `fp == NULL`, the file is missing or inaccessible.
- **EOF** (End Of File) is a special constant used to stop loops.
- Always use **fclose()** after reading.